

Certainly. Below is a **master comparison chart for the major public concrete/general-purpose classes in the Java Collections Framework**, using the comparison dimensions from the previous answer.

A terminology note first: I'll use "**Collections Framework**" **broadly**, including Collection implementations, Map implementations, concurrent collections, legacy collections, and specialized implementations. Map itself does **not** extend Collection.

1. Master list of concrete collection classes

Family	Concrete class	Main interface	Internal structure	Primary purpose
List	ArrayList	List	Resizable array	Fast indexed access
List	LinkedList	List, Deque	Doubly linked list	List + deque operations
Legacy List	Vector	List	Synchronized resizable array	Legacy thread-safe list
Legacy List	Stack	List via Vector	Vector-based stack	Legacy LIFO stack
Set	HashSet	Set	Hash table	Fast unique-element lookup
Set	LinkedHashSet	SequencedSet/Set	Hash table + linked ordering	Unique insertion-ordered elements
Sorted Set	TreeSet	NavigableSet	Red-black tree	Sorted unique elements
Enum Set	EnumSet implementations	Set	Bit-vector-like representation	Efficient enum sets
Queue	PriorityQueue	Queue	Binary heap	Priority processing
Deque	ArrayDeque	Deque	Resizable circular array	Fast queue/stack/deque
Map	HashMap	Map	Hash table	Fast key-value lookup

Map	LinkedHashMap	SequencedMap/Map	Hash table + linked ordering	Ordered key-value map
Sorted Map	TreeMap	NavigableMap	Red-black tree	Sorted keys
Legacy Map	Hashtable	Map	Synchronized hash table	Legacy thread-safe map
Properties	Properties	Map via Hashtable	Hash table	Configuration properties
Identity Map	IdentityHashMap	Map	Hash table using reference equality	Identity-based mappings
Weak Map	WeakHashMap	Map	Hash table + weak keys	GC-sensitive mappings
Enum Map	EnumMap	Map	Array-like enum storage	Enum keys
Concurrent Map	ConcurrentHashMap	ConcurrentMap	Concurrent hash table	Concurrent key-value access
Concurrent Map	ConcurrentSkipListMap	ConcurrentNavigableMap	Skip list	Concurrent sorted map
Concurrent Set	ConcurrentSkipListSet	NavigableSet	Skip list	Concurrent sorted set
Concurrent List	CopyOnWriteArrayList	List	Copy-on-write array	Read-heavy concurrent list
Concurrent Set	CopyOnWriteArraySet	Set	Copy-on-write list	Read-heavy concurrent set
Blocking Queue	ArrayBlockingQueue	BlockingQueue	Fixed-size array	Bounded producer-consumer queue
Blocking Queue	LinkedBlockingQueue	BlockingQueue	Linked nodes	Optional-bounded blocking queue
Blocking Queue	PriorityBlockingQueue	BlockingQueue	Heap	Concurrent priority queue

Blocking Queue	DelayQueue	BlockingQueue	Priority heap	Delayed elements
Blocking Queue	SynchronousQueue	BlockingQueue	Direct handoff	Thread-to-thread transfer
Blocking Deque	LinkedBlockingDeque	BlockingDeque	Linked nodes	Blocking double-ended queue
Transfer Queue	LinkedTransferQueue	TransferQueue	Linked concurrent nodes	Producer-to-consumer transfer

2. Fundamental behavioral comparison

Class	Duplicates	null	Ordering	Sorted?	Indexed?	Thread-safe?
ArrayList	✓	✓	Insertion	✗	✓	✗
LinkedList	✓	✓	Insertion	✗	✓	✗
Vector	✓	✓	Insertion	✗	✓	✓ synchronized
Stack	✓	✓	LIFO usage	✗	✓ inherited	✓ synchronized
HashSet	✗	One null	No guaranteed order	✗	✗	✗
LinkedHashSet	✗	One null	Insertion	✗	✗	✗
TreeSet	✗	Generally ✗	Comparison order	✓	✗	✗
PriorityQueue	✓	✗	Priority	Priority-based	✗	✗

ArrayDeque	✓	✗	Deque order	✗	✗	✗
HashMap	Values yes; keys no	One null key; null values	No guaranteed order	✗	N/A	✗
LinkedHashMap	Values yes; keys no	Yes	Insertion/access order	✗	N/A	✗
TreeMap	Values yes; keys no	Null natural-order key ✗	Key comparison	✓	N/A	✗
Hashtable	Values yes; keys no	✗	No guaranteed order	✗	N/A	✓
IdentityHashMap	Values yes; keys identity-unique	✓	No guaranteed order	✗	N/A	✗
WeakHashMap	Values yes; keys no	✓	No guaranteed order	✗	N/A	✗
EnumMap	Values yes; keys no	Null key ✗	Enum declaration order	✓ by enum order	N/A	✗
ConcurrentHashMap	Values yes; keys no	✗	No guaranteed order	✗	N/A	✓
ConcurrentSkipListMap	Values yes; keys no	✗	Key order	✓	N/A	✓
ConcurrentSkipListSet	✗	✗	Comparison order	✓	✗	✓
CopyOnWriteArrayList	✓	✓	Insertion	✗	✓	✓
CopyOnWriteArraySet	✗	✓	Iteration based on underlying list	✗	✗	✓

3. List classes — detailed comparison

Parameter	ArrayList	LinkedList	Vector	Stack	CopyOnWriteArray List
Main interface	List	List, Deque	List	Legacy stack/List	List
Structure	Dynamic array	Doubly linked nodes	Dynamic array	Vector	Copy-on-write array
Duplicates	Yes	Yes	Yes	Yes	Yes
Null	Yes	Yes	Yes	Yes	Yes
Ordering	Insertion	Insertion	Insertion	LIFO usage	Insertion
Index access	Yes	Yes	Yes	Yes	Yes
Random access	Excellent	Poor	Excellent	Excellent	Excellent
get(index)	$O(1)$	$O(n)$	$O(1)$	$O(1)$	$O(1)$
Append	Amortized $O(1)$	$O(1)$	Amortized $O(1)$	$O(1)^*$	Expensive copy
Beginning insert	$O(n)$	$O(1)$	$O(n)$	N/A	$O(n)$ + copy
Middle insert	$O(n)$	$O(n)$ search	$O(n)$	N/A	$O(n)$ + copy
Search	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Memory	Low/moderate	High	Moderate	Moderate	Higher during writes
Thread-safe	No	No	Yes	Yes	Yes

Iterator	Fail-fast	Fail-fast	Generally fail-fast	Inherited behavior	Snapshot
Best use	General List	Deque + linked operations	Legacy code	Legacy stack code	Read-heavy concurrency

Stack is normally avoided in modern code. Prefer ArrayDeque for ordinary LIFO stack behavior.

4. List examples

ArrayList

```
List<String> languages = new ArrayList<>();
```

```
languages.add("Java");
languages.add("Python");
languages.add("Java");
```

```
System.out.println(languages);
// [Java, Python, Java]
```

```
System.out.println(languages.get(1));
// Python
```

Best when **random/indexed access is important**.

LinkedList

```
LinkedList<String> queue = new LinkedList<>();
```

```
queue.addLast("A");
queue.addLast("B");
queue.addFirst("Start");
```

```
System.out.println(queue);  
// [Start, A, B]
```

Useful when you need the combined List + Deque API, though ArrayDeque is usually preferable when only deque operations are required.

CopyOnWriteArrayList

```
List<String> listeners = new CopyOnWriteArrayList<>();
```

```
listeners.add("Listener1");  
listeners.add("Listener2");
```

```
for (String listener : listeners) {  
    System.out.println(listener);  
}
```

Particularly useful for **many reads and relatively few writes**.

5. Set classes — detailed comparison

Parameter	HashSet	LinkedHashSet	TreeSet	EnumSet	ConcurrentSkipList Set	CopyOnWriteArray Set
Duplicates	No	No	No	No	No	No
Null	One	One	Generally no	No	No	Yes
Order	Unspecified	Insertion	Sorted	Enum order	Sorted	Stable iteration
Sorting	No	No	Yes	Enum declaration	Yes	No

Structure	Hash table	Hash + links	Red-black tree	Bit representation	Skip list	COW list
add()	$O(1)^*$	$O(1)^*$	$O(\log n)$	$O(1)$	$O(\log n)$	$O(n)$ +copy
contains()	$O(1)^*$	$O(1)^*$	$O(\log n)$	$O(1)$	$O(\log n)$	$O(n)$
remove()	$O(1)^*$	$O(1)^*$	$O(\log n)$	$O(1)$	$O(\log n)$	$O(n)$ +copy
Comparator	No	No	Yes	Enum order	Yes	No
Thread-safe	No	No	No	No	Yes	Yes
Best purpose	Fast uniqueness	Ordered uniqueness	Sorted uniqueness	Enum constants	Concurrent sorted set	Read-heavy concurrent set

* means expected/average complexity.

6. HashSet vs LinkedHashSet vs TreeSet

Suppose we insert:

30
10
20
10

HashSet

```
Set<Integer> set = new HashSet<>();
```

```
set.add(30);
```

```
set.add(10);  
set.add(20);  
set.add(10);
```

```
System.out.println(set);
```

Conceptually:

```
10  
20  
30
```

but **do not rely on that iteration order**.

LinkedHashSet

```
Set<Integer> set = new LinkedHashSet<>();
```

```
set.add(30);  
set.add(10);  
set.add(20);  
set.add(10);
```

```
System.out.println(set);
```

Result:

```
[30, 10, 20]
```

TreeSet

```
Set<Integer> set = new TreeSet<>();
```

```
set.add(30);  
set.add(10);
```

```
set.add(20);  
set.add(10);
```

```
System.out.println(set);
```

Result:

[10, 20, 30]

So:

Requirement	Preferred class
Fast unique storage	HashSet
Unique + insertion order	LinkedHashSet
Unique + sorted	TreeSet
Enum values	EnumSet
Concurrent sorted elements	ConcurrentSkipListSet
Concurrent, very read-heavy small set	CopyOnWriteArraySet

7. Queue and Deque classes

Parameter	Priority Queue	Array Deque	Array Blocking Queue	Linked Blocking Queue	Priority Blocking Queue	Delay Queue	Synchronous Queue	Linked Blocking Deque	Linked TransferQueue
FIFO	No	Yes	Yes	Yes	No	No	Handoff	Yes	Generally queue semantics

Priority	Yes	No	No	No	Yes	Delay	No	No	No
Blocking	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Transfer support
Bounded	No	No	Yes	Optional	Effectively unbounded	Unbounded	Zero capacity	Optional	Unbounded
Null	No	No	No	No	No	No	No	No	No
Thread-safe	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Duplicates	Yes	Yes	Yes	Yes	Yes	Yes	N/A storage	Yes	Yes
Main purpose	Priority	Stack/deque	Bounded tasks	Blocking tasks	Priority tasks	Delayed tasks	Handoff	Blocking deque	Transfer

8. Queue method comparison

Operation	Throws exception	Special-value version	Purpose
Insert	add(e)	offer(e)	Insert
Remove	remove()	poll()	Remove head
Examine	element()	peek()	Examine head

For blocking queues, additional operations include:

Method	Purpose
put(e)	Wait until insertion becomes possible
take()	Wait until an element becomes available
offer(e, time, unit)	Timed insertion
poll(time, unit)	Timed retrieval

Example:

```
BlockingQueue<String> jobs =  
    new ArrayBlockingQueue<>(100);
```

```
jobs.put("Job-1");
```

```
String job = jobs.take();
```

This pattern is commonly used in **producer-consumer systems**.

9. ArrayDeque example

```
Deque<String> deque = new ArrayDeque<>();
```

```
deque.addLast("A");  
deque.addLast("B");  
deque.addFirst("START");
```

```
System.out.println(deque);
```

```
// [START, A, B]
```

It can also act as a stack:

```
Deque<String> stack = new ArrayDeque<>();
```

```
stack.push("A");  
stack.push("B");  
stack.push("C");
```

```
System.out.println(stack.pop());  
// C
```

For new code:

```
Stack  
↓  
usually replace with  
↓  
ArrayDeque
```

10. Map classes — master comparison

Although maps belong to the Collections Framework:

Map

does **not** extend:

Collection

They form a separate hierarchy.

Parameter	Hash Map	Linked HashMap	Tree Map	Hashtable	WeakHash Map	Identity HashMap	Enum Map	Concurrent HashMap	Concurrent SkipListMap
Duplicate keys	No	No	No	No	No	Identity-based	No	No	No
Duplicate values	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Null key	Yes	Yes	Generally no	No	Yes	Yes	No	No	No
Null value	Yes	Yes	Yes	No	Yes	Yes	Yes	No	No
Ordering	None guaranteed	Insertion/access	Sorted	None guaranteed	None guaranteed	None guaranteed	Enum order	None guaranteed	Sorted
Sorting	No	No	Yes	No	No	No	Enum order	No	Yes
Structure	Hash table	Hash + links	Red-black tree	Hash table	Weak-key hash	Identity hash	Array-like	Concurrent hash	Skip list
Average get()	O(1)	O(1)	O(log n)	O(1)	O(1)	O(1)	O(1)	O(1)	O(log n)
Thread-safe	No	No	No	Yes	No	No	No	Yes	Yes
Special feature	General map	Predictable order	Sorted keys	Legacy	Weak keys	== semantics	Enum keys	High concurrency	Concurrent sorting

11. Map equality mechanism

This distinction is extremely important.

HashMap

Normally uses:

```
hashCode()  
equals()
```

Example:

```
Map<String, Integer> map = new HashMap<>();
```

```
map.put(new String("Java"), 1);  
map.put(new String("Java"), 2);
```

```
System.out.println(map.size());  
// 1
```

The two strings are logically equal.

IdentityHashMap

Uses reference identity:

```
==
```

rather than normal logical equality.

```
Map<String, Integer> map =  
    new IdentityHashMap<>();
```

```
String a = new String("Java");
```

```
String b = new String("Java");

map.put(a, 1);
map.put(b, 2);

System.out.println(map.size());
// 2
```

Although:

```
a.equals(b)
```

is true,

```
a == b
```

is false.

12. WeakHashMap special behavior

WeakHashMap is unusual because its keys are weakly referenced.

Conceptually:

```
Application
  |
  | strong reference
  v
  Key
  ^
  |
WeakHashMap
weak reference
```

When a key is no longer strongly reachable elsewhere, garbage collection can eventually cause its mapping to disappear.

Example:

```
Map<Object, String> cache = new WeakHashMap<>();
```

```
Object key = new Object();
```

```
cache.put(key, "Metadata");
```

```
key = null;
```

```
// The mapping may later disappear after GC.
```

Possible uses include certain metadata/association structures where entries should not keep keys alive.

13. EnumMap and EnumSet

These specialized collections should be preferred when the domain is an enum.

```
enum Day {  
    MONDAY,  
    TUESDAY,  
    WEDNESDAY,  
    THURSDAY,  
    FRIDAY,  
    SATURDAY,  
    SUNDAY  
}
```

EnumSet

```
Set<Day> weekend =
```

```
EnumSet.of(Day.SATURDAY, Day.SUNDAY);
```

EnumMap

```
Map<Day, String> schedule =  
    new EnumMap<>(Day.class);
```

```
schedule.put(Day.MONDAY, "Java");  
schedule.put(Day.TUESDAY, "Spring");
```

Requirement	Preferred
Set of enum constants	EnumSet
Enum → value mapping	EnumMap

They are generally more compact and efficient for their intended enum-specific use than general-purpose hash/tree alternatives.

14. Concurrent collection comparison

Class	Type	Thread-safe	Ordering	Null	Typical workload
ConcurrentHashMap	Map	Yes	Unspecified	No	Frequent concurrent reads/writes
ConcurrentSkipListMap	Sorted Map	Yes	Sorted	No	Concurrent sorted mappings
ConcurrentSkipListSet	Sorted Set	Yes	Sorted	No	Concurrent sorted elements
CopyOnWriteArrayList	List	Yes	Insertion	Yes	Many reads, few writes
CopyOnWriteArraySet	Set	Yes	Stable iteration	Yes	Many reads, few writes
ArrayBlockingQueue	Queue	Yes	FIFO	No	Bounded producer-consumer

LinkedBlockingQueue	Queue	Yes	FIFO	No	Producer-consumer
PriorityBlockingQueue	Queue	Yes	Priority	No	Concurrent priority processing
DelayQueue	Queue	Yes	Delay	No	Expiring/delayed work
SynchronousQueue	Queue	Yes	Handoff	No	Direct producer-consumer handoff
LinkedBlockingDeque	Deque	Yes	FIFO/deque	No	Blocking both ends
LinkedTransferQueue	Queue	Yes	Queue	No	Direct transfer when possible

15. Copy-on-write mechanism

CopyOnWriteArrayList is fundamentally different from an ordinary synchronized list.

Suppose:

Original

[A][B][C]

A write such as:

```
list.add("D");
```

conceptually creates a new underlying array:

Old snapshot

[A][B][C]

New array

[A][B][C][D]

This makes reads and snapshot iteration convenient, but frequent writes can be expensive.

Therefore:

Workload	Suitability
99% reads, 1% writes	Excellent candidate
Frequent iteration	Excellent
Listener collection	Common use
Frequent add()	Poor choice
Frequent remove()	Poor choice
Large list with constant writes	Poor choice

16. Synchronization comparison

Collection	Synchronization model
ArrayList	None
Vector	Synchronized methods
HashMap	None
Hashtable	Synchronized methods

ConcurrentHashMap	Designed for concurrent access
CopyOnWriteArrayList	Copy-on-write
ConcurrentSkipListMap	Concurrent skip-list algorithms
ArrayBlockingQueue	Blocking synchronization
LinkedBlockingQueue	Blocking synchronization

In modern concurrent code, this:

`ConcurrentHashMap<K,V>`

is generally preferable to using:

`Hashtable<K,V>`

for a new general-purpose concurrent map.

17. Iterator behavior

Collection	Typical iterator behavior
ArrayList	Fail-fast
LinkedList	Fail-fast
HashSet	Fail-fast
LinkedHashSet	Fail-fast

TreeSet	Fail-fast
HashMap	Fail-fast views
TreeMap	Fail-fast views
CopyOnWriteArrayList	Snapshot
ConcurrentHashMap	Weakly consistent
ConcurrentSkipListMap	Weakly consistent
Blocking queues	Generally weakly consistent

Example of a common fail-fast situation:

```
List<String> list =
    new ArrayList<>(List.of("A", "B", "C"));

for (String value : list) {
    list.remove(value);
}
```

This may result in:

ConcurrentModificationException

depending on how modification/traversal proceeds.

Use the iterator's supported operation when appropriate:

```
Iterator<String> iterator = list.iterator();

while (iterator.hasNext()) {
```

```
String value = iterator.next();

if (value.equals("B")) {
    iterator.remove();
}
}
```

18. Comparable vs Comparator

Sorted collections such as:

- TreeSet
- TreeMap
- ConcurrentSkipListSet
- ConcurrentSkipListMap
- PriorityQueue
- PriorityBlockingQueue

need an ordering mechanism.

Comparable

Defines natural ordering inside the class.

```
class Employee implements Comparable<Employee> {

    int id;

    Employee(int id) {
        this.id = id;
    }

    @Override
    public int compareTo(Employee other) {
```

```

    return Integer.compare(id, other.id);
}
}

```

Then:

```

Set<Employee> employees =
    new TreeSet<>();

```

Comparator

Defines ordering externally.

```

Comparator<Employee> byId =
    Comparator.comparingInt(e -> e.id);

```

```

Set<Employee> employees =
    new TreeSet<>(byId);

```

Feature	Comparable	Comparator
Method	compareTo()	compare()
Package	java.lang	java.util
Natural ordering	Yes	No
Custom ordering	Limited to class's natural order	Yes
Multiple strategies	Not naturally	Yes
Modify domain class?	Usually yes	No

19. Time-complexity master chart

The following are the typical/expected complexities useful for choosing a collection; they are not universal worst-case guarantees.

Class	Access	Search	Insert	Remove
ArrayList	$O(1)$	$O(n)$	$O(1)^*$ append	$O(n)$
LinkedList	$O(n)$	$O(n)$	$O(1)^{**}$	$O(1)^{**}$
Vector	$O(1)$	$O(n)$	$O(1)^*$ append	$O(n)$
HashSet	N/A	$O(1)^*$	$O(1)^*$	$O(1)^*$
LinkedHashSet	N/A	$O(1)^*$	$O(1)^*$	$O(1)^*$
TreeSet	N/A	$O(\log n)$	$O(\log n)$	$O(\log n)$
EnumSet	N/A	$O(1)$	$O(1)$	$O(1)$
PriorityQueue	Head $O(1)$	$O(n)$	$O(\log n)$	Head $O(\log n)$
ArrayDeque	Ends $O(1)$	$O(n)$	$O(1)^*$	Ends $O(1)$
HashMap	$O(1)^*$	$O(1)^*$ key	$O(1)^*$	$O(1)^*$
LinkedHashMap	$O(1)^*$	$O(1)^*$ key	$O(1)^*$	$O(1)^*$
TreeMap	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
EnumMap	$O(1)$	$O(1)$	$O(1)$	$O(1)$
ConcurrentHashMap	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(1)^*$
ConcurrentSkipListMap	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

* expected/amortized under normal conditions.

** only after the relevant node/end position is already available; locating an arbitrary index remains $O(n)$.

20. Memory comparison

Class	Relative overhead	Reason
ArrayList	Low	Array of references
LinkedList	High	Object + previous/next references per node
HashSet	Moderate/high	Hash-table entries
LinkedHashSet	Higher	Hash entries + ordering links
TreeSet	High	Tree nodes and links
EnumSet	Very low	Compact bit representation
ArrayDeque	Low	Array-based
HashMap	Moderate	Hash table + entries
LinkedHashMap	Higher	Additional ordering links
TreeMap	High	Tree nodes
EnumMap	Low	Enum-optimized storage
CopyOnWriteArrayList	Potentially high during writes	Array copies
ConcurrentHashMap	Higher than simple map	Concurrency infrastructure

Actual memory consumption depends on JVM implementation, compressed references, table capacity, object layout, element count, load factor, and other runtime factors.

21. Hashing comparison

Hash-based collections include:

HashSet
LinkedHashSet
HashMap
LinkedHashMap
WeakHashMap
ConcurrentHashMap
Hashtable

For ordinary hash-based equality, the core relationship is:

```
hashCode()  
+  
equals()
```

Example:

```
class Employee {  
  
    private final int id;  
  
    Employee(int id) {  
        this.id = id;  
    }  
  
    @Override  
    public boolean equals(Object obj) {
```

```

    if (this == obj)
        return true;

    if (!(obj instanceof Employee other))
        return false;

    return id == other.id;
}

@Override
public int hashCode() {
    return Integer.hashCode(id);
}
}

```

Then:

```
Set<Employee> employees = new HashSet<>();
```

```
employees.add(new Employee(101));
employees.add(new Employee(101));
```

```
System.out.println(employees.size());
// 1
```

22. Capacity and resizing comparison

Class	Capacity concept	Resizing
ArrayList	Internal array	Grows automatically
Vector	Internal array	Grows automatically
HashMap	Hash table capacity	Resizes based on threshold

HashSet	Backing hash table	Resizes
LinkedHashMap	Hash table	Resizes
ArrayDeque	Internal array	Grows
ArrayBlockingQueue	Fixed capacity	Does not dynamically grow
LinkedBlockingQueue	Configurable bound	Node-based
CopyOnWriteArrayList	Array	New array on mutation

For example:

```
List<Integer> values =  
    new ArrayList<>(100_000);
```

can be useful when the approximate required size is already known.

23. Load factor

The load factor is especially important for hash-based collections.

Example:

```
Map<Integer, String> map =  
    new HashMap<>(16, 0.75f);
```

Conceptually:

```
capacity = 16  
loadFactor = 0.75
```

threshold $\approx$
 16×0.75
 $= 12$

Once the relevant threshold is exceeded, the hash table grows/reorganizes.

The default load factor of HashMap is commonly:

0.75

which provides a practical balance between memory usage and lookup efficiency.

24. Sorting comparison

Class	Automatic sorting	Mechanism
ArrayList	No	sort() / Collections.sort()
LinkedList	No	List sorting
HashSet	No	None
LinkedHashSet	No	None
TreeSet	Yes	Comparable/Comparator
PriorityQueue	Priority ordering	Comparable/Comparator
HashMap	No	None
LinkedHashMap	No	Insertion/access order, not sorting

TreeMap	Yes	Key comparison
EnumSet	Enum declaration order	Enum ordinal-based representation/order
EnumMap	Enum declaration order	Enum key order
ConcurrentSkipListSet	Yes	Comparison
ConcurrentSkipListMap	Yes	Key comparison

25. Equality comparison

Different collection families have different equality semantics.

List

Order matters:

```
List<Integer> a = List.of(10, 20, 30);
List<Integer> b = List.of(10, 20, 30);
```

```
System.out.println(a.equals(b));
// true
```

But:

```
List<Integer> b = List.of(30, 20, 10);
```

means:

```
a.equals(b) → false
```

Set

Iteration order does not define Set equality:

```
Set<Integer> a = Set.of(10, 20, 30);  
Set<Integer> b = Set.of(30, 20, 10);
```

```
System.out.println(a.equals(b));  
// true
```

Map

Mappings matter:

```
Map<Integer,String> a =  
    Map.of(1, "A", 2, "B");
```

```
Map<Integer,String> b =  
    Map.of(2, "B", 1, "A");
```

```
System.out.println(a.equals(b));  
// true
```

26. Stream support

Classes implementing Collection inherit:

```
stream()  
parallelStream()
```

Example:

```
List<Employee> employees = new ArrayList<>();
```

```
employees.stream()  
    .filter(e -> e.salary > 50000)
```

```
.map(e -> e.name)
.forEach(System.out::println);
```

Map itself does not have:

```
map.stream()
```

Instead:

```
map.entrySet().stream()
```

or:

```
map.keySet().stream()
```

or:

```
map.values().stream()
```

Example:

```
map.entrySet()
  .stream()
  .filter(e -> e.getValue() > 100)
  .forEach(System.out::println);
```

27. Mutability comparison

Type	Example	Modifiable?
Mutable collection	<code>new ArrayList<>()</code>	Yes
Mutable set	<code>new HashSet<>()</code>	Yes

Mutable map	<code>new HashMap<>()</code>	Yes
Unmodifiable factory list	<code>List.of(...)</code>	No structural modification
Unmodifiable factory set	<code>Set.of(...)</code>	No
Unmodifiable factory map	<code>Map.of(...)</code>	No
Unmodifiable view	<code>Collections.unmodifiableList()</code>	Not through view
Immutable copy-style API	<code>List.copyOf()</code>	No structural modification

Example:

```
List<String> list =
    List.of("Java", "Spring");
```

```
list.add("Hibernate");
```

results in:

`UnsupportedOperationException`

28. Real-world use-case comparison

Requirement	Recommended starting point
General-purpose list	<code>ArrayList</code>
Fast indexed access	<code>ArrayList</code>
List plus deque operations	<code>LinkedList</code>

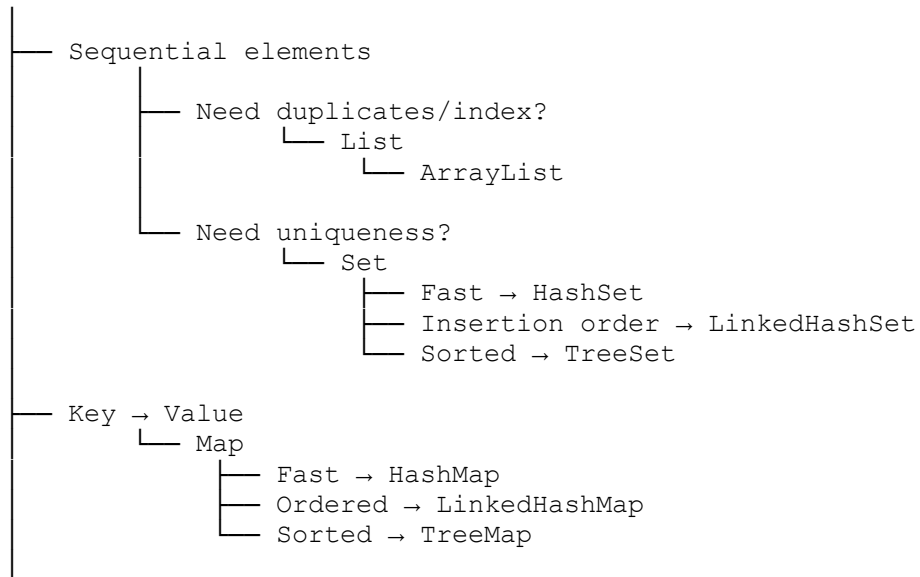
Stack	ArrayDeque
Queue	ArrayDeque
Unique elements	HashSet
Unique + insertion order	LinkedHashSet
Unique + sorted	TreeSet
Enum set	EnumSet
Priority processing	PriorityQueue
General key-value storage	HashMap
Ordered map	LinkedHashMap
Sorted map	TreeMap
Enum-key map	EnumMap
Weak-key association	WeakHashMap
Reference-identity keys	IdentityHashMap
Concurrent map	ConcurrentHashMap
Concurrent sorted map	ConcurrentSkipListMap
Concurrent sorted set	ConcurrentSkipListSet
Read-heavy concurrent list	CopyOnWriteArrayList
Bounded producer-consumer	ArrayBlockingQueue

Blocking producer-consumer	LinkedBlockingQueue
Concurrent priority tasks	PriorityBlockingQueue
Delayed tasks	DelayQueue
Direct thread handoff	SynchronousQueue
Blocking deque	LinkedBlockingDeque
Producer-consumer transfer	LinkedTransferQueue

29. Decision hierarchy

A practical selection process is:

What kind of data?



- └ Processing elements
 - └ Queue / Deque
 - └ Queue/Stack → ArrayDeque
 - └ Priority → PriorityQueue
 - └ Blocking → BlockingQueue implementations

30. Final all-parameter selection matrix

Class	Structure	Dup.	Null	Order	Sorted	Random access	Typical search	Thread-safe	Memory	Primary strength
ArrayList	Array	Yes	Yes	Insert	No	Yes	$O(n)$	No	Low	Random access
LinkedList	Linked	Yes	Yes	Insert	No	Poor	$O(n)$	No	High	List + deque
Vector	Array	Yes	Yes	Insert	No	Yes	$O(n)$	Yes	Medium	Legacy synchronized list
Stack	Vector	Yes	Yes	LIFO	No	Yes	$O(n)$	Yes	Medium	Legacy stack
HashSet	Hash	No	Yes	None	No	No	$O(1)^*$	No	Medium	Unique lookup
LinkedHashSet	Hash+links	No	Yes	Insert	No	No	$O(1)^*$	No	Higher	Ordered uniqueness
TreeSet	Tree	No	Generally no	Sorted	Yes	No	$O(\log n)$	No	High	Sorted uniqueness

EnumSet	Bits	No	No	Enum	Enum	No	$O(1)$	No	Very low	Enum set
PriorityQueue	Heap	Yes	No	Priority	Heap order	No	$O(n)$ arbitrary	No	Low	Priority
ArrayDeque	Array	Yes	No	Deque	No	No	$O(n)$	No	Low	Stack/queue
HashMap	Hash	Values	Yes	None	No	N/A	$O(1)^*$ key	No	Medium	Key lookup
LinkedHashMap	Hash+links	Values	Yes	Insert/access	No	N/A	$O(1)^*$	No	Higher	Ordered map
TreeMap	Tree	Values	Limited key	Sorted keys	Yes	N/A	$O(\log n)$	No	High	Sorted map
Hashtable	Hash	Values	No	None	No	N/A	$O(1)^*$	Yes	Medium	Legacy map
WeakHashMap	Weak hash	Values	Yes	None	No	N/A	$O(1)^*$	No	Medium	Weak keys
IdentityHashMap	Identity hash	Values	Yes	None	No	N/A	$O(1)^*$	No	Medium	Reference equality
EnumMap	Enum array	Values	No key	Enum	Yes	N/A	$O(1)$	No	Low	Enum keys
ConcurrentHashMap	Concurrent hash	Values	No	None	No	N/A	$O(1)^*$	Yes	Medium/high	Concurrent map
ConcurrentSkipListMap	Skip list	Values	No	Sorted	Yes	N/A	$O(\log n)$	Yes	High	Concurrent sorted map

ConcurrentSkip ListSet	Skip list	No	No	Sorted	Yes	No	$O(\log n)$	Yes	High	Concurrent sorted set
CopyOnWrite ArrayList	COW array	Yes	Yes	Insert	No	Yes	$O(n)$	Yes	High on writes	Read-heavy list
CopyOnWrite ArraySet	COW	No	Yes	Stable	No	No	$O(n)$	Yes	High on writes	Read-heavy set
ArrayBlocking Queue	Array	Yes	No	FIFO	No	No	$O(n)$	Yes	Fixed	Bounded queue
LinkedBlocking Queue	Nodes	Yes	No	FIFO	No	No	$O(n)$	Yes	Higher	Blocking queue
PriorityBlocking Queue	Heap	Yes	No	Priority	Priority	No	$O(n)$ arbitrary	Yes	Dynamic	Concurrent priority
DelayQueue	Heap	Yes	No	Delay	Delay	No	—	Yes	Dynamic	Delayed processing
Synchronous Queue	Handoff	N/A	No	Transfer	No	No	N/A	Yes	Very low	Direct handoff
LinkedBlocking Deque	Nodes	Yes	No	Deque	No	No	$O(n)$	Yes	Higher	Blocking deque
LinkedTransfer Queue	Nodes	Yes	No	Queue	No	No	$O(n)$	Yes	Dynamic	Transfer

* indicates expected/average or amortized performance rather than a universal worst-case guarantee.

Most important comparison rule

When you are asked in an **interview, system design discussion, or coding problem** to compare two Java collection implementations, compare them in roughly this order:

Priority	Parameter	Example question
1	Interface/family	List, Set, Queue or Map?
2	Internal data structure	Array, hash table, tree, linked nodes?
3	Duplicate policy	Are duplicates allowed?
4	Null policy	Are null elements/keys/values allowed?
5	Ordering	Insertion, sorted, priority, or unspecified?
6	Random access	Is get(index) efficient?
7	Search complexity	$O(1)$, $O(\log n)$, or $O(n)$?
8	Insert complexity	What does add()/put() cost?
9	Delete complexity	What does remove() cost?
10	Memory	What structural overhead exists?
11	Thread safety	Can concurrent threads use it safely?
12	Iterator behavior	Fail-fast, snapshot, weakly consistent?
13	Equality	equals(), hashing, identity or comparator?

14	Sorting	Comparable/Comparator?
15	Capacity/resizing	Array/hash-table growth?
16	Mutability	Mutable or unmodifiable?
17	Streams	How is stream processing supported?
18	Special behavior	Weak keys, blocking, delay, transfer, COW?
19	Performance workload	Read-heavy, write-heavy, search-heavy?
20	Real-world use case	Why choose this implementation?

The central principle is:

Choose the collection according to semantics first, then performance.

For example, choosing HashSet over ArrayList merely because contains() is typically faster would be incorrect if the application needs duplicates and index-based access.